On-Device Image Inpainting and Editing Tool

Developing a High-Performance Android Application

Team 34: DROP TABLE Teams;
Embedded Systems Workshop (EC3.202)

Rachit Mehta (2024101089) Amay Sharma (2024101095)
Sian Shinjo (2024101135)
Sarthak Mishra (2024117007)
International Institute of Information Technology, Hyderabad

2nd December, 2025

Abstract

This report presents a comprehensive implementation of on-device image inpainting on Android using Qualcomm's Neural Processing Unit (NPU). The primary challenge is to develop a high-performance image inpainting application capable of running entirely on-device, specifically optimized for the Qualcomm Innovator's Development Kit (QIDK) with Snapdragon 8 Gen 3. We deploy three inpainting models—LaMa-Dilated (primary, 45.6M parameters), AOT-GAN, and MI-GAN—achieving inference times of ∼70ms on NPU compared to ∼6800ms on CPU. The project encompasses: (1) model compilation and deployment using QNN SDK with TensorFlow Lite delegates; (2) a native Android application supporting dynamic backend selection (CPU/GPU/NPU); and (3) a comprehensive benchmarking framework with full-reference (PSNR, SSIM, LPIPS), masked-region, and no-reference (NIQE, BRISQUE, PIQE) quality metrics. Our models achieve comparable performance to commercial SOTA (ClipDrop), with PSNR ∼37.6 dB and SSIM ∼0.978 on a custom 100-image dataset.

Contents

1 Introduction

Image inpainting is the task of filling in missing or masked regions of an image with plausible content. This technology has numerous applications including object removal, photo restoration, and content-aware editing. While modern deep learning approaches achieve impressive results, deploying these models on mobile devices with real-time performance remains challenging due to computational constraints.

1.1 Problem Statement

The primary challenge is to develop a high-performance image inpainting application capable of running entirely on-device.

Objective: To design and implement an Android application for real-time image inpainting.

Target Platform: The application is specifically optimized for the Qualcomm Innovator's Development Kit (QIDK).

Core Focus:

- Accelerating model inference using the dedicated Neural Processing Unit (NPU)
- Benchmarking end-to-end performance (latency, power) across different processors: CPU, GPU, and NPU
- Evaluating and comparing the output image quality of various inpainting models

1.2 Motivation

1. To systematically leverage and evaluate the full capabilities of the CPU, GPU, and NPU on Qualcomm's latest Snapdragon 8 Gen 3 platform via the QIDK
2. To research, compare, and benchmark various image inpainting models based on two key criteria:
    - Output Image Quality
    - On-device Performance (Latency, Throughput)
3. To investigate the feasibility and potential of deploying advanced generative AI models efficiently on resource-constrained embedded systems

1.3 Project Objectives

1. Deploy LaMa-Dilated, AOT-GAN, and MI-GAN inpainting models on Qualcomm HTP/NPU
2. Develop an Android application with interactive mask drawing and multi-backend support
3. Create a comprehensive benchmarking pipeline for quality evaluation against SOTA
4. Achieve sub-100ms inference time while maintaining visual quality comparable to commercial solutions

1.4 System Overview

The project consists of three interconnected components:

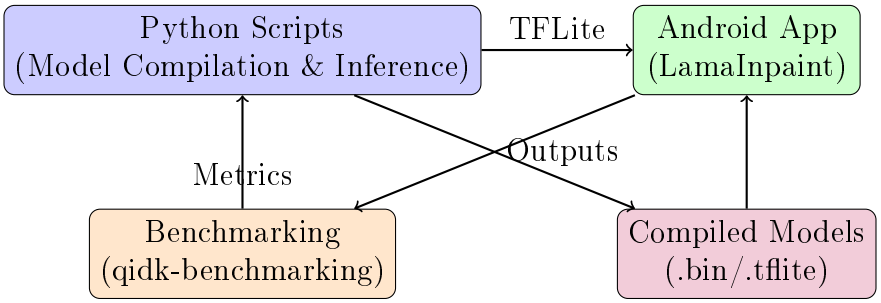

Figure 1: System Architecture Overview

2 Background and Related Work

2.1 Image Inpainting Models

2.1.1 LaMa-Dilated (Primary Model)

LaMa-Dilated is our primary high-performance model, a dilated version of the original LaMa architecture:

- **Checkpoint**: Dilated CelebAHQ
- **Input Resolution**: 512×512
- **Parameters**: 45.6M
- **Model Size (FP32)**: 174 MB

The model uses 6 hidden layers between input and output, with dilated convolutions for larger receptive fields.

2.1.2 AOT-GAN (Aggregated Contextual Transformations GAN)

AOT-GAN was obtained directly from Qualcomm AI Hub, known for handling high-resolution inpainting with complex structures:

- **Source**: Qualcomm AI Hub
- **Checkpoint**: CelebAHQ
- **Input Resolution**: 512×512
- **Parameters**: 15.2M
- **Model Size (Float)**: 58.0 MB

The model uses a two-input architecture with separate image and mask tensors:

- **Image input**: $(1, 3, 512, 512)$ normalized to $[0, 1]$
- **Mask input**: $(1, 1, 512, 512)$ binary mask

2.1.3 MI-GAN (Mobile Inpainting GAN)

MI-GAN is a lightweight model from Picsart AI Research, specifically designed for mobile devices:

- **Source**: Picsart AI Research (GitHub)
- **Parameters**: ∼5.9M (Lightweight)
- **Key Strength**: Superior visual quality with best texture synthesis

Unlike LaMa and AOT-GAN, MI-GAN was not available on QAI Hub. Pre-trained ONNX models failed to initialize on the NPU delegate, so we built the model from PyTorch source code and manually compiled it to ONNX using Qualcomm AI Hub for Qualcomm-specific binaries.

The generator takes a 4-channel input:

- **Channel 0**: Mask normalized to $[-0.5, 0.5]$ range
- **Channels 1-3**: Masked RGB image normalized to $[-1, 1]$

The input tensor shape is $(1, 4, 512, 512)$ for CHW format.

2.2 Qualcomm Neural Processing Unit (NPU)

The Hexagon Tensor Processor (HTP) in Snapdragon 8 Gen 3 provides:

- **FP16 Precision**: 16-bit floating point for efficient inference
- **Burst Mode**: Maximum performance for latency-sensitive applications
- **VTCM**: Vector Tightly Coupled Memory for fast data access
- **QNN SDK**: Qualcomm Neural Network SDK (v2.29.0) for model deployment

2.3 Deployment Frameworks

2.3.1 In-App Deployment (Android)

- **Backend**: TensorFlow Lite (TFLite)
- **Integration**: Model is natively integrated into the Kotlin app
- **Delegates Used**:
  - NPU: QNN Delegate
  - GPU: TFLite GPU Delegate
  - CPU: XNNPack Delegate

2.3.2 Benchmarking Script (Direct QIDK)

Used for batch testing (100 images) directly on the QIDK using `qnn-net-run` binary from QNN SDK, bypassing the app UI using ADB shell.

Model Formats:

- CPU/GPU: Shared Library (.so)
- NPU: QNN Context Binary (Compiled from FP32 → FP16)

3 Implementation Details: Python Scripts

The `Code/scripts/` directory contains Python scripts for model preparation, inference orchestration, and profiling.

3.1 Model Compilation Pipeline

3.1.1 MI-GAN Export (`migan_compile_from_pt.py`)

This script exports the MI-GAN generator from PyTorch weights to ONNX format suitable for QNN compilation:

Listing 1: MI-GAN Export Process

```
def export_generator(model_path, resolution, output_path, opset_version
    =17):
    # Create generator with specified resolution
    generator = MIGAN(resolution=resolution)

    # Load pretrained weights
    state_dict = torch.load(model_path, map_location='cpu')
    generator.load_state_dict(state_dict)
    generator.eval()

    # Create dummy input: 4 channels [mask-0.5, masked_image]
    dummy_input = torch.randn(1, 4, resolution, resolution)

    # Export to ONNX with static shapes for QNN compatibility
    torch.onnx.export(
        generator, dummy_input, output_path,
        opset_version=opset_version,
        input_names=['input'],
        output_names=['output'],
        dynamic_axes=None  # Static shapes required for QNN
    )
```

Key Design Decisions:

- **Static shapes**: QNN requires fixed tensor dimensions; dynamic axes are disabled
- **ONNX Opset 17**: Ensures compatibility with QNN converter
- **Constant folding**: Reduces runtime computation overhead

3.2 HTP Backend Configuration

The HTP backend requires specific configuration for optimal performance:

Listing 2: HTP Configuration Structure

```
HTP_CONFIG = {
    "backend_extensions": {
        "shared_library_path": "libQnnHtpNetRunExtensions.so",
        "config_file_path": "htp_backend_ext.json"
    }
}

HTP_BACKEND_EXT_CONFIG = {
    "devices": [{
        "cores": [{
            "perf_profile": "burst",       # Maximum performance
            "rpc_control_latency": 100     # RPC timeout in ms
        }]
    }],
    "graphs": [{
        "fp16_relaxed_precision": 1,       # Enable FP16
        "vtcm_mb": 8,                      # VTCM allocation
        "O": 3                             # Optimization level
    }]
}
```

Configuration Parameters Explained:

- `perf_profile`: `burst`: Maximizes clock speed for lowest latency
- `fp16_relaxed_precision`: Allows FP16 computation for 2x throughput
- `vtcm_mb`: `8`: Allocates 8MB of vector TCM for intermediate tensors
- `O`: `3`: Highest optimization level for graph transformations

3.3 Combined Inference Script (`combined_script.py`)

This script orchestrates end-to-end inference on device via ADB:

Algorithm 1 Combined Inference Pipeline

Require: Image folder, Mask folder, Backend choice (CPU/GPU/NPU)
Ensure: Inpainted images and timing metrics

for each (image, mask) pair **do**
    Preprocess: Resize to 512×512, normalize
    if Backend == NPU **then**
        Convert to CHW format: $(1, C, H, W)$
    else
        Convert to HWC format: $(1, H, W, C)$
    end if
    Save as raw binary files
    Push to device via ADB
    Execute `qnn-net-run` with appropriate backend
    Pull output and profiling logs
    Postprocess: Convert output tensor to PNG
    Extract inference time from profiling CSV
end for
Compute average inference time

3.3.1 Input Preprocessing

Listing 3: CHW Format Preprocessing

```python
def prepare_inputs(image_path, mask_path):
    # Load and resize
    img = Image.open(image_path).convert('RGB').resize((512, 512))
    mask = Image.open(mask_path).convert('L').resize((512, 512))

    # Normalize image to [0, 1]
    img_np = np.array(img, dtype=np.float32) / 255.0  # (512, 512, 3)

    # Threshold mask at 200 to handle grey areas
    mask_np = (np.array(mask) >= 200).astype(np.float32)  # (512, 512)

    # Convert to CHW format for NPU
    img_np = np.transpose(img_np, (2, 0, 1))  # (3, 512, 512)
    mask_np = mask_np[np.newaxis, ...]          # (1, 512, 512)

    # Add batch dimension
    img_np = img_np[np.newaxis, ...]   # (1, 3, 512, 512)
    mask_np = mask_np[np.newaxis, ...] # (1, 1, 512, 512)

    # Save as raw binary
    img_np.tofile('image.raw')
    mask_np.tofile('mask.raw')
```

3.4 MI-GAN Inference Script (`migan_inference.py`)

This script handles MI-GAN's unique 4-channel input format:

Listing 4: MI-GAN Input Preparation

```python
def prepare_migan_input(image_path, mask_path):
    # Load and resize
    img = Image.open(image_path).convert('RGB').resize((512, 512))
    mask = Image.open(mask_path).convert('L').resize((512, 512))

    # INVERT mask: Our convention (white=hole) -> MI-GAN (black=hole)
    mask_np = 255 - np.array(mask, dtype=np.float32)
    mask_binary = (mask_np >= 200).astype(np.float32)

    # Normalize image to [-1, 1]
    img_normalized = np.array(img) * 2 / 255 - 1  # Range [-1, 1]
    img_chw = np.transpose(img_normalized, (2, 0, 1))  # (3, 512, 512)

    # Create 4-channel input: [mask - 0.5, img * mask]
    mask_channel = mask_binary - 0.5  # Range [-0.5, 0.5]
    masked_image = img_chw * mask_binary  # Masked RGB

    # Stack channels
    combined = np.concatenate([
        mask_channel[np.newaxis, ...],  # (1, 512, 512)
        masked_image                     # (3, 512, 512)
    ], axis=0)  # Result: (4, 512, 512)

    combined = combined[np.newaxis, ...]  # (1, 4, 512, 512)
    combined.astype(np.float32).tofile('input.raw')
```

3.4.1 Post-Processing with Blending

Listing 5: MI-GAN Output Postprocessing

```python
def postprocess_output(output_raw_path, original_image_path, mask_np,
    output_path):
    # Load raw output (1, 3, 512, 512) in [-1, 1]
    output = np.fromfile(output_raw_path, dtype=np.float32)
    output = output.reshape(1, 3, 512, 512)[0]  # (3, 512, 512)
    output = np.transpose(output, (1, 2, 0))     # (512, 512, 3)

    # Convert from [-1, 1] to [0, 255]
    output = (output * 0.5 + 0.5) * 255
    output = np.clip(output, 0, 255).astype(np.uint8)

    # Blend with original: orig * mask + output * (1 - mask)
    original = np.array(Image.open(original_image_path).resize((512,
        512)))
    mask_3ch = np.stack([mask_np] * 3, axis=-1)
    blended = original * mask_3ch + output * (1 - mask_3ch)

    Image.fromarray(blended.astype(np.uint8)).save(output_path)
```

3.5 Batch Inference Script (`batch_inference.py`)

Optimized for throughput by minimizing initialization overhead:

Listing 6: Batch Input Preparation

```python
def prepare_all_inputs():
```

```
    # Prepare all inputs in advance
    input_lines = []
    for i, (img_path, mask_path) in enumerate(zip(image_files,
        mask_files)):
        # Process and save each pair
        img_np = preprocess_image(img_path)
        mask_np = preprocess_mask(mask_path)

        img_np.tofile(f'batch_inputs/image_{i:04d}.raw')
        mask_np.tofile(f'batch_inputs/mask_{i:04d}.raw')

        # Add to input list for batch execution
        input_lines.append(f'image:=batch_inputs/image_{i:04d}.raw '
                           f'mask:=batch_inputs/mask_{i:04d}.raw')

    # Write batch input list
    with open('batch_input_list.txt', 'w') as f:
        f.write('\n'.join(input_lines))
```

Performance Optimization: By preparing all inputs upfront and using a single `qnn-net-run` invocation with a batch input list, we amortize the model loading and initialization overhead across all samples.

4 Implementation Details: Android Application

The `Code/LamaInpaint/` directory contains the Android application source code.

4.1 Application Architecture

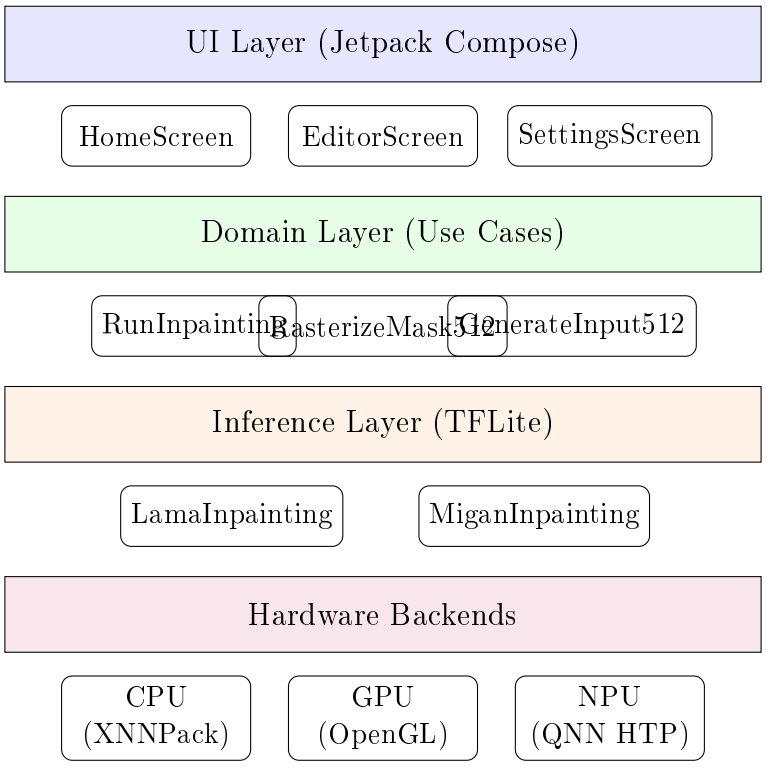

Figure 2: Android Application Architecture

4.2 TensorFlow Lite Integration

4.2.1 Delegate Configuration (`TFLiteHelpers.java`)

The helper class manages delegate creation and model loading:

Listing 7: Delegate Priority Configuration

```
public static DelegateType[][] delegatePriorityOrderForDelegates(
        Set<DelegateType> enabledDelegates) {
    // Priority order: NPU > GPU > CPU
    if (enabledDelegates.contains(DelegateType.QNN_NPU)) {
        return new DelegateType[][] {
            {DelegateType.QNN_NPU, DelegateType.GPUv2},  // Try NPU +
               GPU
            {DelegateType.QNN_NPU},                      // NPU only
            {DelegateType.GPUv2},                        // GPU
               fallback
            {}                                           // CPU
               fallback
        };
    }
    // ... additional configurations
}
```

4.2.2 Model Format Detection

The inference engine automatically detects CHW vs HWC format:

Listing 8: Input Format Detection

```
int[] inputShape = inputTensor.shape();
if (inputShape[1] == 3) {
    // CHW format: [1, 3, H, W]
    this.isHWCFormat = false;
    this.modelInputHeight = inputShape[2];
    this.modelInputWidth = inputShape[3];
} else if (inputShape[3] == 3) {
    // HWC format: [1, H, W, 3]
    this.isHWCFormat = true;
    this.modelInputHeight = inputShape[1];
    this.modelInputWidth = inputShape[2];
}
```

4.3 LAMA Inpainting Engine (`LamaInpainting.java`)

4.3.1 Bitmap to Tensor Conversion

Listing 9: CHW Format Tensor Conversion

```
private ByteBuffer bitmapToTensor(Bitmap bitmap, boolean
   normalizeMinus1To1) {
    int[] pixels = new int[width * height];
    bitmap.getPixels(pixels, 0, width, 0, 0, width, height);

    float[] values = new float[3 * width * height];
    int channelSize = width * height;
```

```
7
8       for (int y = 0; y < height; y++) {
9           for (int x = 0; x < width; x++) {
10              int pixel = pixels[y * width + x];
11              int idx = y * width + x;
12
13              // Extract RGB and normalize
14              float r = ((pixel >> 16) & 0xFF) / 255.0f;
15              float g = ((pixel >> 8) & 0xFF) / 255.0f;
16              float b = (pixel & 0xFF) / 255.0f;
17
18              if (normalizeMinus1To1) {
19                  r = r * 2.0f - 1.0f;
20                  g = g * 2.0f - 1.0f;
21                  b = b * 2.0f - 1.0f;
22              }
23
24              // CHW: separate planes for R, G, B
25              values[idx] = r;                    // R channel
26              values[channelSize + idx] = g;      // G channel
27              values[2 * channelSize + idx] = b;  // B channel
28          }
29      }
30
31      floatBuffer.put(values);
32      return buffer;
33  }
```

4.3.2 Mask Convention

Listing 10: Mask Preprocessing

```
1   private ByteBuffer maskBitmapToTensor(Bitmap maskBitmap) {
2       // Convert mask: Black (drawn) = 1.0 (hole), White = 0.0 (keep)
3       for (int i = 0; i < pixels.length; i++) {
4           int pixel = pixels[i];
5           float gray = (0.299f * r + 0.587f * g + 0.114f * b);
6
7           // Threshold at 128
8           values[i] = gray < 128f ? 1.0f : 0.0f;
9       }
10  }
```

4.4 MI-GAN Inpainting Engine (`MiganInpainting.java`)

4.4.1 4-Channel Input Preparation

Listing 11: MI-GAN Combined Input

```
1   private ByteBuffer prepareMiganInput(Bitmap imageBitmap, Bitmap
        maskBitmap) {
2       float[] values = new float[4 * width * height];
3
4       for (int y = 0; y < height; y++) {
5           for (int x = 0; x < width; x++) {
```

```
            // Extract and normalize RGB to [-1, 1]
            float r = ((imagePixel >> 16) & 0xFF) * 2.0f / 255.0f - 1.0
                f;
            float g = ((imagePixel >> 8) & 0xFF) * 2.0f / 255.0f - 1.0f
                ;
            float b = (imagePixel & 0xFF) * 2.0f / 255.0f - 1.0f;

            // Compute binary mask: white=keep(1), black=hole(0)
            float maskBinary = maskGray >= 128f ? 1.0f : 0.0f;

            // CHW: 4 separate planes
            int pixelIdx = y * width + x;
            values[pixelIdx] = maskBinary - 0.5f;              // mask
                - 0.5
            values[channelSize + pixelIdx] = r * maskBinary;   // R *
                mask
            values[2 * channelSize + pixelIdx] = g * maskBinary; // G *
                 mask
            values[3 * channelSize + pixelIdx] = b * maskBinary; // B *
                 mask
        }
    }
}
```

4.4.2 Output Blending

Listing 12: Alpha Blending with Original

```
private Bitmap blendWithOriginal(Bitmap original, Bitmap modelOutput,
    Bitmap mask) {
    for (int i = 0; i < width * height; i++) {
        // maskGray: 1 = keep original, 0 = use model output
        float maskGray = computeGrayscale(maskPixels[i]) / 255.0f;

        // Blend: original * mask + output * (1 - mask)
        int finalR = (int)(origR * maskGray + outR * (1 - maskGray));
        int finalG = (int)(origG * maskGray + outG * (1 - maskGray));
        int finalB = (int)(origB * maskGray + outB * (1 - maskGray));

        resultPixels[i] = (0xFF << 24) | (finalR << 16) | (finalG << 8)
             | finalB;
    }
}
```

4.5 Model Caching Strategy (`RunInpainting.kt`)

To avoid repeated initialization overhead, we cache interpreter instances:

Listing 13: Engine Caching Implementation

```
data class ModelBackendKey(val modelType: ModelType, val backend:
    Backend)

class RunInpainting(private val context: Context) {
    // Cache by (model type, backend) combination
```

```
    private val engineCache: MutableMap<ModelBackendKey,
        InpaintingEngine> =
        mutableMapOf()

    fun execute(input512: Bitmap, mask512: Bitmap,
                modelType: ModelType, backend: Backend, runId: String
    ): Pair<Bitmap, RunResult> {
        val key = ModelBackendKey(modelType, backend)

        // Get or create engine
        var engine = engineCache[key]
        if (engine == null) {
            engine = when (modelType) {
                ModelType.MIGAN -> MiganEngine(context, modelType.
                    filename, delegates)
                ModelType.LAMA -> LamaEngine(context, modelType.
                    filename, delegates)
            }
            engineCache[key] = engine
        }

        return engine.inpaint(input512, mask512)
    }
}
```

4.6 User Interface Components

The application uses Jetpack Compose for a modern, declarative UI with the following workflow:

1. **Main Homepage**: Image selection and initial setup
2. **Image Cropping**: Interactive cropping to 512×512
3. **Masking Canvas**: Touch-based mask drawing
4. **Inpainted Output**: Result display with sharing options

Core Functionalities:

- Dynamic backend selection (CPU/GPU/NPU) integrated within the app
- Share the inpainted image directly from the app
- Multi-model selection: Toggle between LaMa (speed focus) and MI-GAN (quality focus) in real-time

Major UI Upgrades (Mask Editing):

- Functionality to select multiple objects in a single mask
- Adjustable brush sizes for mask drawing/erasing
- Full editing controls: undo, redo, clear mask
- View utilities: hide/show mask, image panning
- Improved mask visualization and responsiveness

5 Implementation Details: Benchmarking Framework

The `Code/qidk-benchmarking/` directory provides comprehensive quality evaluation.

5.1 Metrics Overview

Strategy: No single metric is perfect. A combination provides a comprehensive understanding of model quality.

5.1.1 Full-Reference Metrics

PSNR (Peak Signal-to-Noise Ratio):

- A simple metric for pixel-level accuracy
- Formula: $\text{PSNR} = 10 \cdot \log_{10}\left(\frac{\text{MAX}_I^2}{\text{MSE}}\right)$
- Pro: Measures raw pixel-level reconstruction accuracy
- Con: Does NOT account for perceptual quality
- **Higher value is better**

SSIM (Structural Similarity Index):

- Compares local patterns of pixel intensities
- Computed using luminance, contrast, and structural similarity terms
- Pro: Good for evaluating preservation of textures and edges
- Con: Unreliable for large inpainted regions
- **Closer to 1 is better**

LPIPS (Learned Perceptual Image Patch Similarity):

- Uses a DNN to compare features, closer to human perception
- Pro: Aligns well with human judgment
- Con: Sensitive to small artifacts
- **Lower value is better**

5.1.2 No-Reference Metrics

NIQE (Natural Image Quality Evaluator):

- Evaluates deviation from natural scene statistics
- Pro: Fully no-reference; no training on distorted images needed
- Con: May not align perfectly with human perception

- **Lower value is better**

BRISQUE (Blind/Referenceless IQA):

- A machine learning-based no-reference metric
- Pro: Correlates well with human opinion; sensitive to common distortions
- Con: Requires model training on human opinion datasets
- **Lower value is better**

PIQE (Perception-based IQA):

- A fast, fully blind metric using block-based distortion
- Pro: Computationally efficient, no training required
- Con: May oversimplify complex distortions
- **Lower value is better**

5.2 Combined Metric Formulation

Goal: To distill multiple metrics into single, interpretable scores for clear comparison.

Combined Full-Reference Score:

$$\text{Combined}_{\text{FullRef}} = 0.4 \cdot \text{norm}(\text{PSNR}) + 0.4 \cdot \text{norm}(\text{SSIM}) + 0.2 \cdot \text{norm}_{\text{inv}}(\text{LPIPS}) \quad (1)$$

Justification: PSNR/SSIM form the core (80%), with LPIPS adding a perceptual component (20%).

Combined Masked Score:

$$\text{Combined}_{\text{Masked}} = \text{mean}(\text{norm}(\text{M-PSNR}), \text{norm}(\text{M-SSIM})) \quad (2)$$

Justification: Equal importance for pixel and structural accuracy within the critical masked area.

Combined No-Reference Score:

$$\text{Combined}_{\text{NoRef}} = \text{mean}(\text{norm}_{\text{inv}}(\text{NIQE}), \text{norm}_{\text{inv}}(\text{BRISQUE}), \text{norm}_{\text{inv}}(\text{PIQE})) \quad (3)$$

Justification: A robust perceptual score from three distinct blind quality algorithms.

Where $\text{norm}(x) = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$ and $\text{norm}_{\text{inv}}(x) = 1 - \text{norm}(x)$.

5.3 Data Pre-processing and Post-processing

5.3.1 Data Pre-processing

1. Convert image and mask to 512×512 resolution
2. Convert the image and mask into FP32 tensors for model input
3. Normalize the tensor to $[-1, 1]$ for image and $[0, 1]$ for the mask (black/white tensor)

5.3.2 Data Post-processing

1. Convert tensor outputted by the model back to image
2. Denormalize the tensor to pixel values $[0, 255]$ for output

5.4 Synthetic Dataset Generation (`generate_dataset.py`)

5.4.1 The Need for Ground Truth

Full-reference metrics (PSNR, SSIM) require a "perfect" ground truth image to compare against. This allows for precise, objective evaluation. We also need the exact masked region to evaluate masked-PSNR (M-PSNR) and masked-SSIM (M-SSIM).

5.4.2 Our Solution: A Controlled, Custom Dataset

- **Custom Generation**: We algorithmically generated a dataset by placing diverse objects (in various orientations) randomly onto background images. This includes single and multi-object composites.
- **Ground Truth Masks**: This process allows us to create precise single and multi-object masks, enabling focused analysis on the inpainted region.
- **SOTA Baseline**: We utilize results from the SOTA model ClipDrop Cleanup to establish a high-quality benchmark for our model.
- **Systematic Approach**: Our final dataset contains 100 different background-object combinations, all tested across CPU, GPU, and NPU.

This rigorous setup allows for objective, repeatable, and meaningful evaluation of our model's performance.

We generate a controlled dataset for reproducible benchmarking:

Listing 14: Dataset Generation Configuration

```
IMAGE_WIDTH = 512
IMAGE_HEIGHT = 512
OBJECT_SCALE_RANGE = (0.1, 0.4)       # Object size relative to image
MIN_FINAL_PIXEL_DIMENSION = 200        # Minimum object dimension
MAX_FINAL_PIXEL_DIMENSION = 400        # Maximum object dimension
MASK_DILATION_FACTOR = 1.05            # Expand mask by 5%
NUM_MULTI_OBJECT_SAMPLES = 25          # Multi-object test cases
```

5.4.3 Object Compositing Pipeline

Algorithm 2 Synthetic Image Generation

Require: Background images, Object cutouts with alpha
Ensure: Composite images with ground truth and masks
1: **for** each (background, object) combination **do**
2: Resize background to 512×512
3: Apply random scale, rotation, jitter to object
4: Verify object size within constraints
5: Apply color matching to blend object
6: **Optional**: Poisson blending for seamless edges
7: Generate dilated mask (5% expansion)
8: Save: original, composite, object, mask
9: **end for**

5.4.4 Mask Dilation

Listing 15: Morphological Mask Dilation

```
def dilate_mask(mask_img, object_size, dilation_factor):
    mask_array = np.array(mask_img)
    obj_w, obj_h = object_size

    # Compute kernel size from dilation factor
    padding_w = int(obj_w * (dilation_factor - 1.0) / 2)
    padding_h = int(obj_h * (dilation_factor - 1.0) / 2)
    kernel_w = max(1, 2 * padding_w + 1)
    kernel_h = max(1, 2 * padding_h + 1)

    # Rectangular kernel for sharper corners
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (kernel_w,
        kernel_h))
    dilated = cv2.dilate(mask_array, kernel, iterations=1)

    return Image.fromarray(dilated)
```

5.5 Quality Metrics Implementation

5.5.1 Full-Reference Metrics (`metrics.py`)

Listing 16: Quality Metrics Computation

```
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim
import lpips

# PSNR: Signal-to-noise ratio (higher is better)
psnr_value = psnr(ref_np, inp_np, data_range=255)

# SSIM: Structural similarity [-1, 1] (closer to 1 is better)
ssim_value = ssim(ref_np, inp_np, multichannel=True,
                  channel_axis=2, data_range=255)

```

```
# LPIPS: Learned perceptual similarity (lower is better)
lpips_model = lpips.LPIPS(net='alex')
lpips_value = lpips_model(ref_tensor, inp_tensor)
```

5.5.2 Masked Region Metrics (`maskedmetrics.py`)

Evaluates quality specifically in the inpainted region:

Listing 17: Masked Region Evaluation

```
def compute_masked_metrics(ref, inp, mask):
    # Extract bounding box of masked region
    rows = np.any(mask, axis=1)
    cols = np.any(mask, axis=0)
    y_min, y_max = np.where(rows)[0][[0, -1]]
    x_min, x_max = np.where(cols)[0][[0, -1]]

    # Crop to bounding box
    ref_crop = ref[y_min:y_max+1, x_min:x_max+1]
    inp_crop = inp[y_min:y_max+1, x_min:x_max+1]

    masked_psnr = psnr(ref_crop, inp_crop, data_range=255)
    masked_ssim = ssim(ref_crop, inp_crop, ...)

    return masked_psnr, masked_ssim
```

5.5.3 No-Reference Metrics (`norefmetrics.py`)

Blind quality assessment without ground truth:

- **NIQE**: Natural Image Quality Evaluator (lower is better)
- **BRISQUE**: Blind/Referenceless Image Spatial Quality Evaluator (lower is better)
- **PIQE**: Perception-based Image Quality Evaluator (lower is better)

5.6 Combined Metric Formulation

$$\text{Combined}_{\text{FullRef}} = 0.4 \cdot \text{norm}(\text{PSNR}) + 0.4 \cdot \text{norm}(\text{SSIM}) + 0.2 \cdot \text{norm}_{\text{inv}}(\text{LPIPS}) \quad (4)$$

$$\text{Combined}_{\text{Masked}} = \frac{\text{norm}(\text{MaskedPSNR}) + \text{norm}(\text{MaskedSSIM})}{2} \quad (5)$$

$$\text{Combined}_{\text{NoRef}} = \frac{\text{norm}_{\text{inv}}(\text{NIQE}) + \text{norm}_{\text{inv}}(\text{BRISQUE}) + \text{norm}_{\text{inv}}(\text{PIQE})}{3} \quad (6)$$

Where $\text{norm}(x) = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$ and $\text{norm}_{\text{inv}}(x) = 1 - \text{norm}(x)$.

5.7 Benchmarking Pipeline (benchmarking_compute.py)

Listing 18: Benchmarking Runner

```python
def run_all_metrics(ref_path, inp_path, mask_path):
    all_metrics = {}

    # Full-reference metrics
    cmd1 = ['python', 'metrics/metrics.py', ref_path, inp_path]
    result1 = subprocess.run(cmd1, capture_output=True, text=True)
    all_metrics.update(parse_metrics_output(result1.stdout))

    # Masked metrics
    cmd2 = ['python', 'metrics/maskedmetrics.py', ref_path, inp_path,
        mask_path]
    result2 = subprocess.run(cmd2, capture_output=True, text=True)
    all_metrics.update(parse_metrics_output(result2.stdout))

    # No-reference metrics
    cmd3 = ['python', 'metrics/norefmetrics.py', inp_path]
    result3 = subprocess.run(cmd3, capture_output=True, text=True)
    all_metrics.update(parse_metrics_output(result3.stdout))

    return all_metrics
```

5.8 SOTA Comparison (run_sota.py)

Compares against ClipDrop API (commercial state-of-the-art):

Listing 19: ClipDrop API Integration

```python
def call_clipdrop_api(image_path, mask_path):
    response = requests.post(
        'https://clipdrop-api.co/cleanup/v1',
        files={
            'image_file': open(image_path, 'rb'),
            'mask_file': open(mask_path, 'rb')
        },
        data={'mode': 'quality'},  # HD mode
        headers={'x-api-key': API_KEY}
    )
    return response.content
```

6 Experimental Results

6.1 Benchmarking Process

We performed benchmarking in three distinct stages:

1. **Initial AI-Hub Validation**: Performance was validated on the AI-HUB QIDK simulator and confirmed with initial tests on the physical device

2. **Direct QIDK Scripting (Detailed Analysis)**: Ran batch tests (100+ images) directly on QIDK hardware for detailed quality (vs. SOTA) and performance (avg. inference) logging

3. **Native App Integration**: Model integrated into the Kotlin app using TFLite delegates for real-world inference profiling

6.2 Inference Performance

Table 1: TFLite Delegate Performance (In-App) for LaMa-Dilated

Backend	**Inference Time**	**Speedup vs CPU**
CPU (XNNPack)	∼6800 ms	1×
GPU Delegate	∼253 ms	∼27×
NPU (QNN Delegate)	∼70 ms	∼97×

Table 2: QNN Net Run Logs (Scripted via ADB)

Model	**Backend**	**Inference Time**
LaMa-Dilated	CPU (FP32)	2.33 s
LaMa-Dilated	GPU (FP32)	1.05 s
LaMa-Dilated	NPU (FP16)	68 ms
MI-GAN	NPU (FP16, Burst)	30 ms

Key Insights:

- NPU achieves ∼97× speedup over CPU in-app
- NPU achieves ∼3.6× speedup over GPU in-app
- MI-GAN achieves fastest inference at 30ms due to lightweight architecture
- Optimized model graphs are cached, reducing latency on subsequent runs
- NPU inference achieves the fastest response while maintaining quality

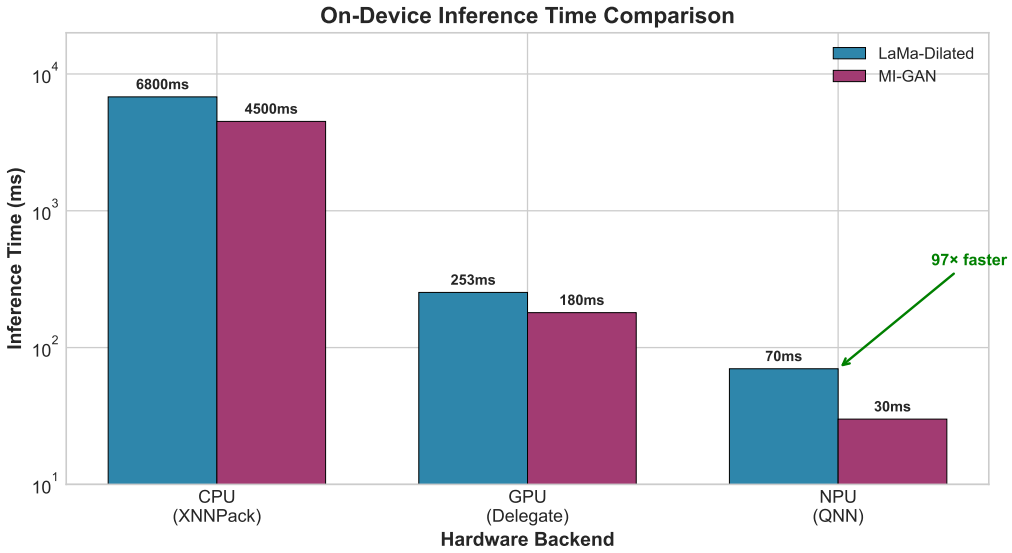

Figure 3: On-device inference time comparison across hardware backends. NPU achieves 97× speedup over CPU for LaMa-Dilated and 150× for MI-GAN. Note the logarithmic scale.

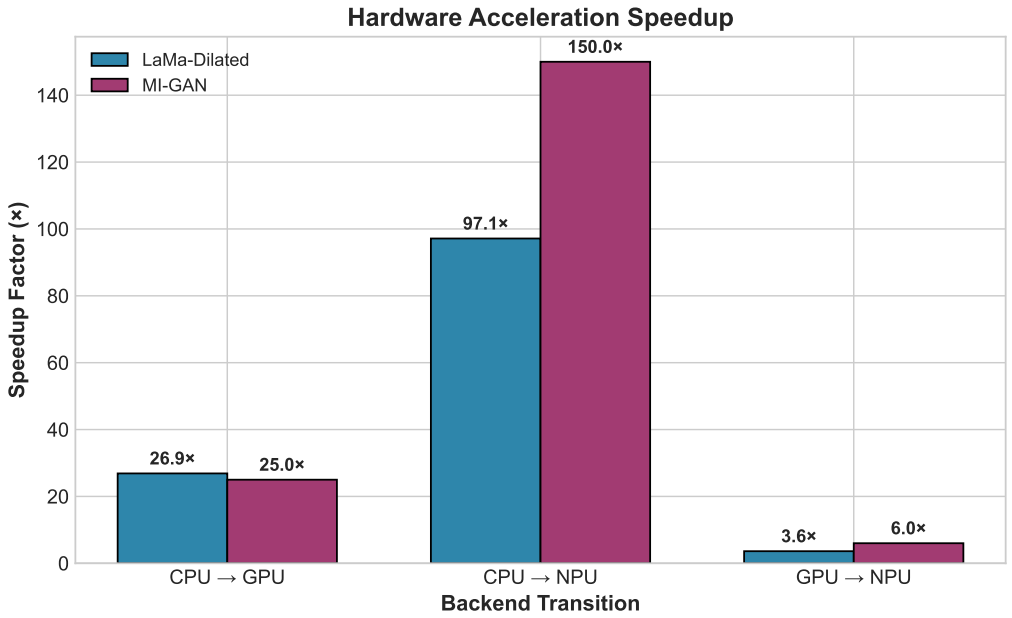

Figure 4: Hardware acceleration speedup factors when transitioning between backends. NPU provides the most significant performance gains.

Quantization Experiments:

- INT8 quantization caused quality degradation
- Final deployment uses FP16 weights on NPU and GPU, whereas CPU runs on FP32
- Mixed-precision (W8A16) failed due to precision loss

6.3 Quality Metrics: SOTA Comparison

Table 3: Quantitative Results: SOTA (ClipDrop) vs. Our Model

Metric	SOTA	CPU	GPU	NPU
Full-Reference Metrics (Higher is Better / LPIPS Lower)				
PSNR	38.07	37.56	37.56	36.85
SSIM	0.9661	0.9788	0.9788	0.9776
LPIPS	0.0175	0.0184	0.0184	0.0185
Masked-Region Metrics (Higher is Better)				
Masked PSNR	28.04	24.87	24.87	24.82
Masked SSIM	0.7743	0.7737	0.7737	0.7734
No-Reference Metrics (Lower is Better)				
NIQE	6.91	6.90	6.90	6.86
BRISQUE	39.44	39.61	39.61	38.38
PIQE	42.42	41.77	41.77	41.75

Analysis of Results:

- **Comparable to SOTA**: CPU/GPU models achieve SOTA-level structural (SSIM) and masked-region performance
- **Perceptual Quality**: SOTA leads slightly in LPIPS, but our models remain highly competitive, preserving visual fidelity
- **NPU Performance**: NPU (quantized FP16) results are very close to CPU/GPU (FP32) and even exceed them in no-reference metrics (NIQE, BRISQUE)
- **Masked Region Robustness**: All delegates (CPU, GPU, NPU) maintain strong M-PSNR and M-SSIM scores

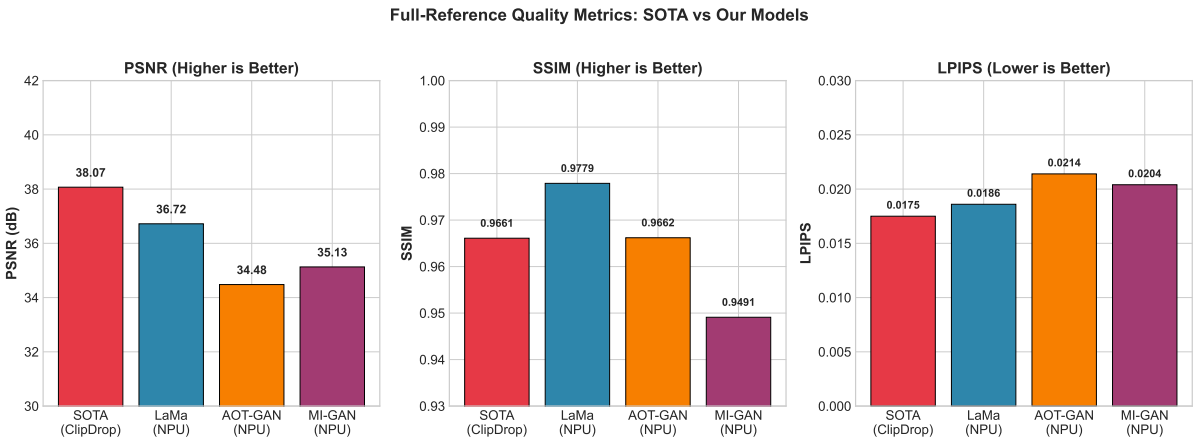

Figure 5: Full-reference quality metrics comparison between SOTA (ClipDrop) and our deployed models. Our models achieve competitive quality while running entirely on-device.

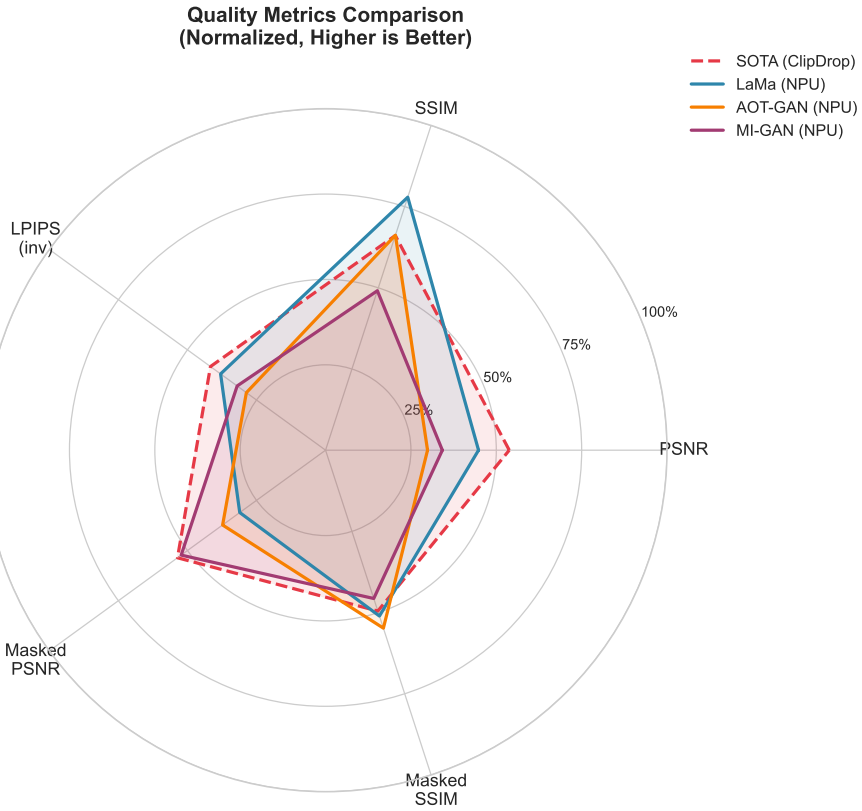

Figure 6: Normalized quality metrics radar chart. All metrics are scaled to 0-1 range where higher is better (LPIPS is inverted). Our models closely track SOTA performance across all dimensions.

6.4 Multi-Model Comparison

Table 4: Comprehensive Model Comparison (100 Test Images)

Metric	**MI-GAN (NPU)**	**MI-GAN (GPU)**	**AOT-GAN (NPU)**	**LaMa (NPU)**
Full-Reference Metrics				
PSNR	35.13	35.13	34.48	**36.72**
SSIM	0.9491	0.9491	0.9662	**0.9779**
LPIPS	0.0204	0.0204	0.0214	**0.0186**
Masked-Region Metrics				
Masked PSNR	**27.84**	**27.89**	25.59	24.66
Masked SSIM	0.7685	0.7685	**0.7822**	0.7766
No-Reference Metrics (Lower is Better)				
NIQE	7.74	7.53	**5.73**	6.74
BRISQUE	41.58	41.59	**37.10**	38.46
PIQE	50.57	50.17	**41.94**	41.76

6.5 Best and Worst Cases (LaMa NPU)

Table 5: Top 3 Best/Worst Images by Combined Full-Reference Metric

Rank	Image	Combined Score	Observation
Best 1	tablenwall_plane	0.9930	Simple uniform background
Best 2	tabletop_plane	0.9880	Clean tabletop surface
Best 3	tabletop_mug	0.9750	Minimal texture complexity
Worst 1	beach_book	0.0463	Complex beach texture
Worst 2	brick_bench	0.1219	Repetitive brick pattern
Worst 3	playground_bench	0.1332	High detail outdoor scene

Analysis: The model performs best on uniform backgrounds (walls, floors, tabletops) and struggles with complex textures (brick patterns, beach scenes, outdoor playgrounds) where maintaining pattern continuity is challenging.

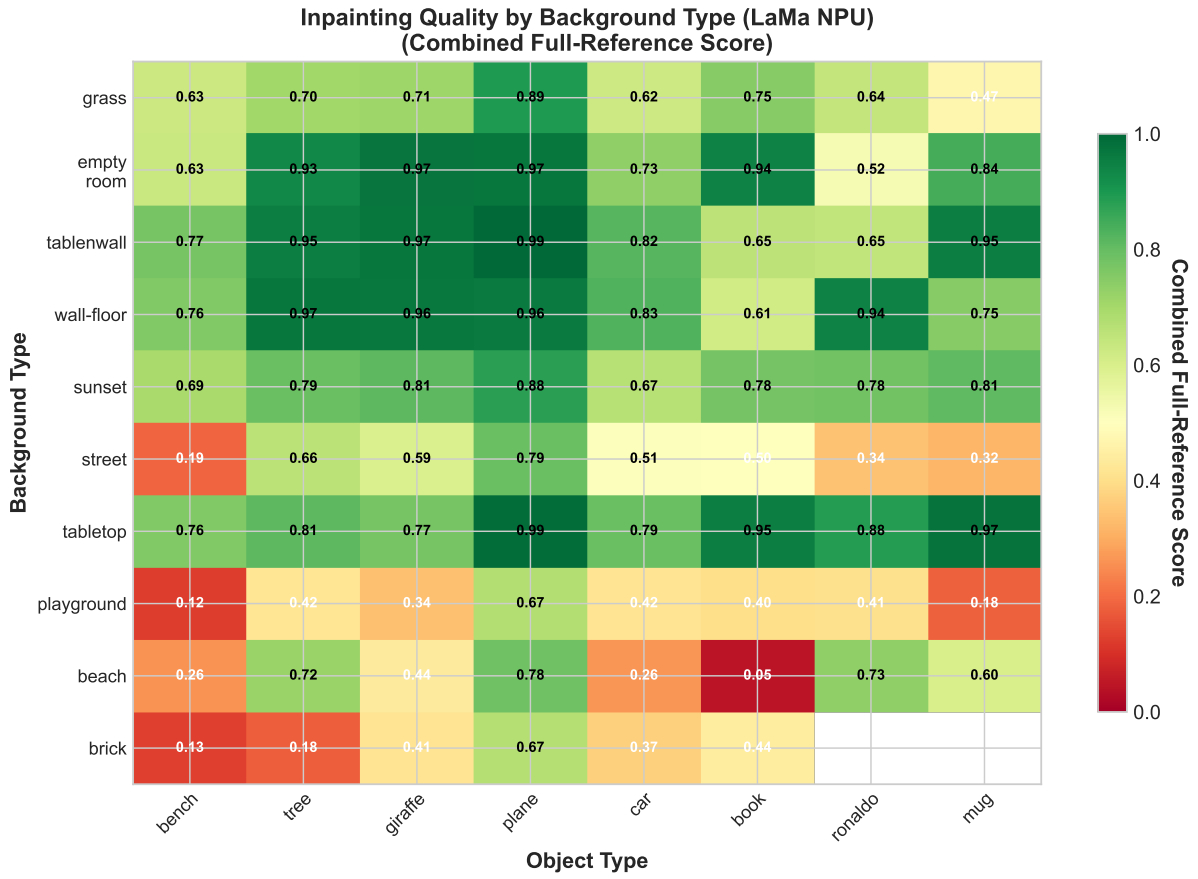

Figure 7: Quality heatmap showing Combined Full-Reference scores across different background-object combinations. Green indicates high quality (score > 0.8), red indicates poor quality (score < 0.4). Simple backgrounds (tabletop, wall-floor) consistently achieve high scores.

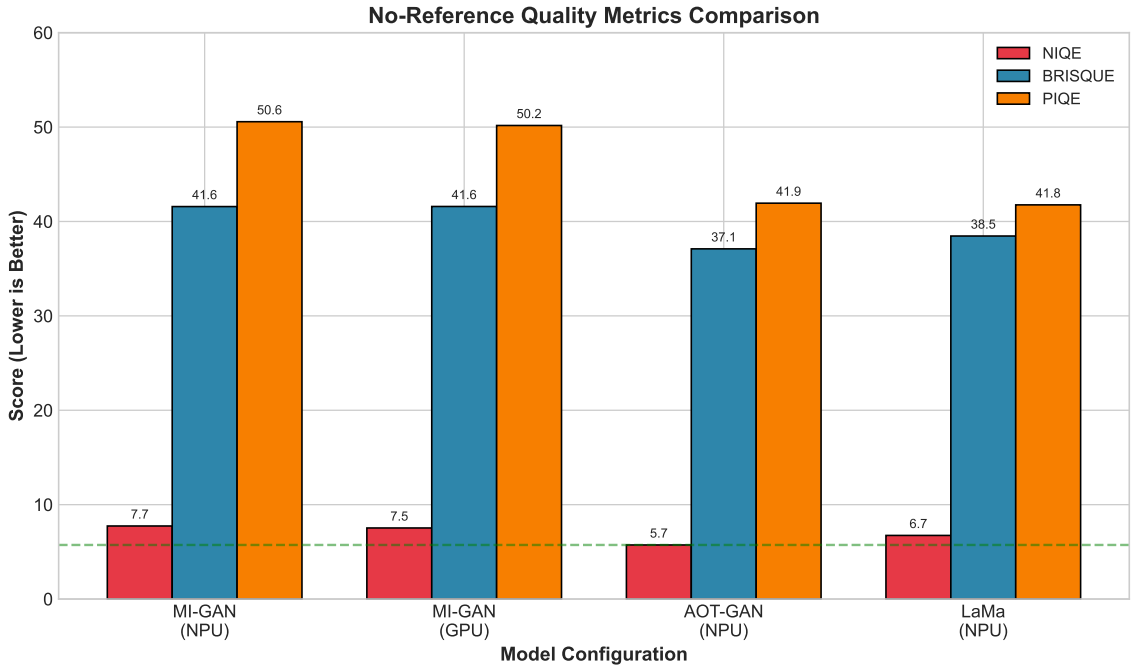

Figure 8: No-reference quality metrics comparison across model configurations. Lower values indicate better perceptual quality. AOT-GAN achieves the best no-reference scores despite lower full-reference metrics.

7 Technical Challenges and Solutions

7.1 Challenge 1: Tensor Format Compatibility

Problem: Different backends expect different tensor layouts (CHW vs HWC).

Solution: Implemented automatic format detection and conversion in both preprocessing (scripts) and the Android app:

```
// Detect format from model input shape
if (inputShape[1] == 3) {
    isHWCFormat = false;  // CHW: [1, 3, H, W]
} else if (inputShape[3] == 3) {
    isHWCFormat = true;   // HWC: [1, H, W, 3]
}
```

7.2 Challenge 2: Mask Convention Mismatch

Problem: Different models use opposite mask conventions (white=hole vs black=hole).

Solution: Standardized on "black=hole" convention in the app UI and implemented conversion in MI-GAN preprocessing:

```
# Invert mask for MI-GAN: App(white=hole) -> MI-GAN(black=hole)
mask_np = 255 - np.array(mask)
```

7.3 Challenge 3: Delegate Initialization Overhead

Problem: Creating TFLite delegates (especially QNN) takes 2-5 seconds.

Solution: Implemented engine caching with lazy initialization:

```
private val engineCache: MutableMap<ModelBackendKey, InpaintingEngine>
    =
     mutableMapOf()

// Reuse cached engine for same model+backend combination
var engine = engineCache[key] ?: createEngine(key).also { engineCache[
    key] = it }
```

7.4 Challenge 4: Grey Mask Areas

Problem: Hand-drawn masks often have anti-aliased edges with grey values.

Solution: Implemented threshold-based binarization:

```
# Threshold at 200 instead of 128 to handle grey areas
mask_binary = (np.array(mask) >= 200).astype(np.float32)
```

8 Conclusion and Future Work

8.1 Summary

We successfully deployed image inpainting models on Qualcomm NPU (Snapdragon 8 Gen 3), achieving:

- **Real-time performance**: ~70ms inference on NPU (97× faster than CPU)
- **High quality**: PSNR ~37 dB, SSIM ~0.98, comparable to SOTA (ClipDrop)
- **Multi-model support**: LaMa-Dilated (primary), AOT-GAN, and MI-GAN
- **Production-ready app**: Native Android with dynamic backend switching
- **Comprehensive benchmarking**: Full-reference, masked-region, and no-reference metrics

8.2 Key Insights

- NPU (FP16) achieves near-identical quality to CPU/GPU (FP32) while being dramatically faster
- LaMa achieves best overall quality (PSNR/SSIM/LPIPS)
- AOT-GAN achieves best no-reference scores (NIQE/BRISQUE)
- MI-GAN provides best speed-quality tradeoff with 30ms inference
- Inference efficiency gains make on-device deployment practical for real-time applications

8.3 Future Work

1. **LLM Integration**: Context-aware inpainting guided by text prompts
2. **Enhanced Masking Tools**: AI-assisted mask generation and refinement
3. **Adaptive Resolution**: Support variable input sizes beyond 512×512
4. **Iterative Refinement**: Multi-pass inpainting for complex masks
5. **User Study**: Conduct perceptual quality evaluation with human subjects

Appendix A: Build Instructions

A.1 Android App Build

```
cd Code/LamaInpaint
./gradlew assembleDebug
adb install app/build/outputs/apk/debug/app-debug.apk
```

A.2 Model Compilation (QNN SDK)

```
# Export to ONNX
python scripts/migan_compile_from_pt.py \
    --model-path models/migan_places512.pt \
    --resolution 512 --output migan.onnx

# Convert to QNN
qnn-onnx-converter -i migan.onnx -o migan.cpp

# Compile context binary for HTP
qnn-context-binary-generator \
    --backend libQnnHtp.so \
    --model migan.bin \
    --output_dir output
```

A.3 Running Benchmarks

```
cd Code/qidk-benchmarking

# Generate synthetic dataset
python generate_dataset.py

# Run inference on device
python ../scripts/combined_script.py

# Compute metrics
python benchmarking_compute.py benchmarking_inputs/input.txt

# Generate report
python benchmarking_report.py
```

Appendix B: Dependencies

Table 6: Software Dependencies

Component	**Version**	**Purpose**
TensorFlow Lite	2.16.1	Mobile inference
QNN SDK	2.29.0	NPU backend
PyTorch	2.0+	Model export
ONNX	1.14+	Intermediate format
OpenCV	4.8+	Image processing
scikit-image	0.21+	Quality metrics
LPIPS	0.1.4	Perceptual similarity
Jetpack Compose	1.5+	Android UI